

Top 200 Git Debug Commands



By Devops Shack

[Click here for DevSecOps and Cloud DevOps Course](#)

DevOps Shack

Top 200 Git Debug Commands

- `git status -sb` – Short + branch; clutter-free snapshot.
- `git diff` – What changed in working tree vs index.
- `git diff --staged` – What's staged vs HEAD.
- `git diff HEAD` – All local changes vs last commit.
- `git diff --name-only` – Only filenames changed.
- `git diff --word-diff` – Word-level changes for noisy lines.
- `git diff --stat` – Change summary (files/lines).
- `git show` – Show a commit + patch; quick inspect.
- `git show --name-only <rev>` – Files touched in a commit.
- `git show --stat <rev>` – Stats of a commit.
- `git show :path/to/file` – Show staged blob.
- `git log` – History of current branch.
- `git log --oneline --graph --decorate --all` – Compact graph for branch tangle.
- `git log -p` – Patch view; see exactly what changed.
- `git log --stat` – Per-commit stats to spot big changes.
- `git log --name-status` – File list + status (A/M/D).
- `git log --follow -- <file>` – History of a renamed/moved file.
- `git log -S 'needle'` – History where a string was added/removed.

-
- `git log -G 'regex'` – History where a regex matched diffs.
 - `git log --author='<name|email>'` – Filter by author (ownership).
 - `git log --grep='message text'` – Filter by message (bug IDs).
 - `git log --since='2 weeks ago'` – Time-bounded history.
 - `git log --merges` – See merge commits; trace integration points.
 - `git log --no-merges` – Focus on actual changes, not merges.
 - `git blame <file>` – Who/when changed each line (root cause hunt).
 - `git blame -L 50,120 <file>` – Blame a line range.
 - `git blame -w <file>` – Ignore whitespace when blaming.
 - `git annotate <file>` – Alias of blame in some installs.
 - `git bisect start` – Begin binary search for bad commit.
 - `git bisect bad` – Mark current as bad (reproduces bug).
 - `git bisect good <rev>` – Mark a known good point.
 - `git bisect run <test.sh>` – Automate pass/fail search.
 - `git bisect reset` – Exit bisect and return to original HEAD.
 - `git fsck` – Check repo integrity (dangling/lost objects).
 - `git fsck --lost-found` – Save lost objects in `.git/lost-found`.
 - `git gc` – Clean up and optimize; can fix odd slowness.
 - `git gc --aggressive` – Heavier optimization (use sparingly).
 - `git count-objects -vH` – Size/loose objects breakdown.
 - `git rev-parse <thing>` – Resolve refs to SHA (scripts).
 - `git rev-parse --is-inside-work-tree` – Detect repo correctly.
 - `git rev-parse --abbrev-ref HEAD` – Current branch name.
 - `git cat-file -p <sha>` – Pretty-print objects (commit/tree/blob).

-
- `git cat-file -t <sha>` – Type of object (commit/tree/blob/tag).
 - `git cat-file -s <sha>` – Object size (debug large blobs).
 - `git ls-tree <rev>` – List tree (directory snapshot) of a commit.
 - `git ls-tree -r <rev>` – Recursive listing; find missing paths.
 - `git show-ref` – Show refs and SHAs (debug weird refs).
 - `git for-each-ref` – Scriptable ref inspection (filter/sort).
 - `git name-rev <sha>` – Map SHA to nearest ref/branch.
 - `git describe --tags` – Humanish name for a commit (nearest tag).
 - `git describe --contains <sha>` – Which tag contains a commit.
 - `git check-ignore -v <path>` – Why a file is ignored.
 - `git check-attr -a -- <path>` – Show .gitattributes applied.
 - `git check-ref-format <ref>` – Validate ref names (CI hooks).
 - `git symbolic-ref HEAD` – Which ref HEAD points to.
 - `git reflog` – All HEAD moves; recover lost commits.
 - `git reflog show <branch>` – Branch movement history.
 - `git show HEAD@{yesterday}` – State of HEAD at a time.
 - `git reset --hard <reflog-entry>` – Recover to a known point (careful).
 - `git stash push -u -m "msg"` – Save dirty state fast (incl. untracked).
 - `git stash list` – See stashes (debug "lost" changes).
 - `git stash show -p stash@{0}` – Patch of a stash.
 - `git stash drop stash@{0}` – Delete a specific stash.
 - `git stash pop` – Reapply and drop (may conflict).
 - `git clean -nd` – Dry-run which untracked files would be removed.
 - `git clean -fd` – Force remove untracked dirs/files (danger).

-
- `git merge-base A B` – Best common ancestor for conflict analysis.
 - `git cherry -v A B` – Commits in A not in B; compare branches.
 - `git cherry-pick <rev>` – Bring one commit; debugging regression fixes.
 - `git revert <rev>` – Safe undo via inverse commit.
 - `git revert -m 1 <merge-commit>` – Revert a merge (parent pick).
 - `git reset --hard` – Reset index + working tree to HEAD (danger).
 - `git reset --keep <rev>` – Move HEAD but keep working changes if possible.
 - `git reset --soft <rev>` – Move HEAD only; keep index+work (rebuild commit).
 - `git restore <path>` – Discard unstaged changes in file.
 - `git restore --staged <path>` – Unstage without losing edits.
 - `git switch <branch>` – Safer branch switch (than checkout).
 - `git switch -c <new>` – Create + switch (debug ephemeral branches).
 - `git worktree list` – See all linked worktrees (multi-checkout).
 - `git worktree add ../wt <branch>` – Parallel working copy for debugging.
 - `git worktree remove ../wt` – Remove a worktree cleanly.
 - `git grep 'needle'` – Search tracked files (fast code search).
 - `git grep -nI 'needle'` – Show line numbers, skip binaries.
 - `git grep -e 'one' -e 'two'` – Multiple patterns.
 - `git grep --cached 'needle'` – Search index (staged content).
 - `git grep <pat> <rev> -- <path>` – Search in a past tree.
 - `git show :0:<file>` – Show merge index (during conflicts).
 - `git ls-files` – Show tracked files; debug staging.
 - `git ls-files -s` – Show stage info (resolve conflicts by stage).
 - `git ls-files -u` – Unmerged entries (conflict details).

-
- `git update-index --refresh` – Re-stat files if status looks wrong.
 - `git update-index --assume-unchanged <file>` – Ignore local tweaks (perf).
 - `git update-index --no-assume-unchanged <file>` – Undo above.
 - `git update-index --skip-worktree <file>` – Keep local changes out of index.
 - `git update-index --no-skip-worktree <file>` – Undo skip-worktree.
 - `git config --list --show-origin` – All configs + where they came from.
 - `git config --global -e` – Edit global config; fix bad settings.
 - `git config --local -e` – Edit repo config.
 - `git config --system -e` – Edit system config (admin).
 - `git remote -v` – Remotes + URLs; debug wrong origin.
 - `git remote show origin` – Fetch/push refs; prune status.
 - `git remote prune origin` – Remove stale tracking branches.
 - `git fetch` – Update refs without merging; safe inspect.
 - `git fetch --all --prune` – Sync everything, drop stale.
 - `git fetch origin <branch>` – Update just one branch.
 - `git pull --ff-only` – Avoid unintended merge commits.
 - `git pull --rebase` – Keep linear history (safer review).
 - `git push --force-with-lease` – Safe-ish force; protects others' work.
 - `git push -u origin <branch>` – Set upstream (fix tracking errors).
 - `git branch -vv` – Local branches + upstream + tip commit.
 - `git branch --merged` – What's already merged (cleanup).
 - `git branch --no-merged` – What's not merged (pending).
 - `git branch -r` – Remote tracking branches.
 - `git branch -a` – All branches (local+remote).

-
- `git show-branch --more=20` – Compare branch heads quickly.
 - `git rebase -i <base>` – Interactively clean/fixup messy history.
 - `git rebase --rebase-merges` – Preserve merges while rebasing.
 - `git rebase --abort` – Stop a broken rebase.
 - `git rebase --continue` – Continue after fixing conflicts.
 - `git rerere status` – See recorded conflict resolutions.
 - `git rerere diff` – Show how conflicts were auto-resolved.
 - `git rerere gc` – Clean old rerere records.
 - `git clean -Xdf` – Remove ignored files only (cache reset).
 - `git clean -xdf` – Remove all untracked + ignored (reset sandbox).
 - `git archive <rev> | tar -x` – Extract a clean tree to inspect outside repo.
 - `git verify-commit <sha>` – GPG/signature check for a commit.
 - `git verify-tag <tag>` – Verify signed tag (release integrity).
 - `git tag -n` – Tags with first line messages.
 - `git tag -l 'v*' --sort=-v:refname` – Version-sorted tags.
 - `git show <tag>` – Show tagged commit and notes.
 - `git notes show <rev>` – Extra annotations (if used).
 - `git shortlog -sn` – Top committers; ownership patterns.
 - `git shortlog -sn --since='30 days ago'` – Recent authors; hotspot debugging.
 - `git blame --show-email <file>` – Exact emails (mailmap issues).
 - `git mailmap` (via `.mailmap`) – Normalize author identities.
 - `git show --pretty=fuller <rev>` – Author/committer times contrast (rebases).
 - `git show --pretty=raw <rev>` – Raw headers for low-level debugging.
 - `git log --decorate --simplify-by-decoration --all` – Top refs only view.

-
- `git log --first-parent` – History along mainline only (release audit).
 - `git log --cherry-pick --right-only A...B` – Unique commits on B vs A.
 - `git range-diff A...B` – Compare two series of commits (post-rebase).
 - `git show -m <merge-commit>` – Show per-parent diffs for a merge.
 - `git diff --merge-base A B` – Diff from best common base.
 - `git blame --reverse <file>` – Track introduction of lines.
 - `git blame <file> <rev>` – Blame at a historic revision.
 - `git restore --source=<rev> <file>` – Bring file from another commit (debug regression).
 - `git checkout --detach <sha>` – Inspect old snapshot without moving branches.
 - `git clean -nX` – Dry-run ignore-only cleanup.
 - `git clean -nx` – Dry-run remove ignored and untracked.
 - `git show --pretty=format:'%h %ad %s' --date=short` – Custom, concise commit view.
 - `git log --abbrev-commit` – Short SHAs for quick eyeballing.
 - `git log -n 1 -- <file>` – Last change to a file.
 - `git whatchanged -- <file>` – Old school per-file history (similar to `log -p`).
 - `git show --quiet <sha>` – Only metadata, no patch.
 - `git blame --porcelain <file>` – Script-friendly blame output.
 - `git ls-remote origin` – See remote refs without fetching (CI sanity).
 - `git remote get-url origin` – Current URL (SSH/HTTPS mixups).
 - `git remote set-url origin <url>` – Fix wrong remote URL.
 - `git submodule status` – Submodule SHAs; drift detection.
 - `git submodule summary` – Pending commits in submodules.
 - `git submodule update --init --recursive` – Fix “missing submodule content”.
 - `git submodule sync --recursive` – Align URLs after remote changes.

-
- `git blame --ignore-revs-file .git-blame-ignore-revs` – Ignore reformat commits.
 - `git range-diff --creation-factor=80` – Better rename/creation sensitivity.
 - `git diff --find-renames` – Detect renames in diffs.
 - `git diff --find-copies` – Detect copied files (slower).
 - `git log --diff-filter=D` – Find deletions.
 - `git log --diff-filter=R` – Find renames.
 - `git log --diff-filter=T` – Type changes (file↔dir).
 - `git log --diff-filter=ACMRT` – Include only these change types.
 - `git blame -C -C <file>` – Detect moved code across files.
 - `git blame -M <file>` – Detect moved code within a file.
 - `git show --no-patch --pretty=full <rev>` – Commit metadata only.
 - `git pack-objects` (advanced) – Inspect pack creation (performance labs).
 - `git verify-pack -v .git/objects/pack/*.idx` – Find huge/bad objects.
 - `git unpack-objects < pack` – Low-level inspection (rare).
 - `git replace <old> <new>` – Temporarily substitute objects (debug history).
 - `git replace -l` – List replacements.
 - `git branch --contains <sha>` – Branches that contain a commit.
 - `git tag --contains <sha>` – Tags that contain a commit.
 - `git blame --line-porcelain -L <start>,<end> <file>` – Machine-readable slice.
 - `git log --pretty=format:'%h %D %s'` – Show ref names per commit (where it's pointed).
 - `git fetch --depth=1` – Shallow fetch to isolate current state (CI speed debug).
 - `git rev-list --count HEAD` – Number of commits (sanity metric).
 - `git rev-list --max-parents=0 HEAD` – Root commit(s).
 - `git show --raw` – Low-level diff of a commit (paths, modes).

-
- `git ls-files --others --exclude-standard` – True untracked set.
 - `git restore --source=HEAD --staged --worktree .` – Hard restore to HEAD (safer than reset for trees).
 - `git show :1:<file>` – Base version during a merge conflict.
 - `git show :2:<file>` – Ours side in conflict (current branch).
 - `git show :3:<file>` – Theirs side in conflict (merged-in).
 - `git merge --abort` – Back out of a merge in trouble.
 - `git cherry-pick --abort` – Stop a broken cherry-pick.
 - `git cherry-pick --continue` – Continue after resolving conflicts.
 - `GIT_TRACE=1 git <cmd>` – Trace high-level Git calls to stderr.
 - `GIT_TRACE_PACKET=1 git fetch` – Trace wire protocol (network debug).
 - `GIT_CURL_VERBOSE=1 git fetch` – Verbose HTTP(S) transfers.
 - `git bugreport` – Auto-collect environment/repo info for support.
 - `git diagnose` (*if available in your Git*) – Produce a diagnostic bundle for debugging issues.